SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Name	S. Mukesh Tharun
Register Number	1971260006
Course Code & Name	C Programming / CSA02
Slot	CSA02
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	CSA02-1971260006D-Assignment1
Submission Date	27-10-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 – Understand the Problem and Identify the Data

Task / Problem Statement

The Grama Jyothi Rural Electricity Co-operative serves approximately 4,000 households monthly at the village level, with scalability projections extending to 60,000 households (taluk level) and 800,000 households (district level). Meter readers record the previous reading and current reading for each household. Incorrect legacy spreadsheets resulted in discrepancies including billing errors off by several paise, negative bills caused by unhandled 16-bit meter register rollovers (from 65535 to 0), and fragmented tariff revisions.

Aim

To analyze the billing problem for the Grama Jyothi Rural Electricity Co-operative and identify every item of data the program must store and compute, ensuring absolute accuracy to the paisa across district-level scaling.

Concept / Theory

Understanding electrical billing requires analyzing inputs (previous reading, current reading, household number, category) and outputs (units consumed, energy charge, fixed charge, total bill). Invariant parameters like tariff slabs and rates are defined as constants. Using unsigned arithmetic prevents negative rollover anomalies, while storing monetary values in integer paise avoids floating-point precision drift.

Procedure / Algorithm

1. Analyze the operational scale and requirements of the Grama Jyothi Rural Electricity Co-operative (4,000 to 800,000 households).

2. Identify all input parameters (previous reading, current reading, household ID, category) and output parameters (units, charges, total).

3. Determine invariant parameters that must be established as symbolic constants.

4. Formulate correctness rules (no negative bills, sum of parts equals total, paisa exactness).

5. Analyze alternative representations such as string-based readings and evaluate their time and space complexities.

Data Item Identification & Justification Table

Data Item	Type / Category	Justification & Range
Previous Reading	Input (Unsigned)	The meter reading recorded at the end of the previous billing cycle. Unsigned integer ranging from 0 to 65535.

Current Reading	Input (Unsigned)	The meter reading recorded at the end of the current billing cycle. Unsigned integer ranging from 0 to 65535.
Household Number	Input (Integer)	Unique identifier assigned to each consumer connection (e.g., 1 to 800,000 for district scale).
Consumer Category	Input (Character/Enum)	Classification of electricity usage (e.g., Domestic, Commercial, Agricultural).
Units Consumed	Computed (Integer)	Calculated total electricity consumed during the billing cycle (Current - Previous, with rollover handling).

Constants, Negative/Fractional Values, and Billing Rules

• Constants: Tariff slab limits (SLAB1_UNITS, SLAB2_UNITS), energy rates (RATE1_PAISE, RATE2_PAISE, RATE3_PAISE), and fixed charges (FIXED_CHARGE_PAISE) remain entirely invariant during program execution and are defined via preprocessor macros.

• Negative, Fractional, and Large Values: Meter readings and consumption cannot be negative when computed using unsigned arithmetic. Money is never stored as floating-point rupees (to avoid IEEE-754 precision drift) but as integer paise.

• Correctness Rules: (1) A consumer bill can never be negative. (2) The total bill must strictly equal the sum of the fixed charge and all energy slab charges. (3) Calculations must be exact to the paisa.

• Assumptions: Standard 16-bit mechanical meter registers (max value 65535) are deployed across households.

Alternative Concept: Readings Stored as Text (Character Strings)

If meter readings were stored as character strings (e.g., "01530") instead of numeric integers, the solution would require string-to-integer conversion routines (such as atoi or manual character parsing). Comparing or computing consumption would necessitate processing each character from left to right (proportional to digit count d). In terms of space, storing a 16-bit integer requires exactly 2 bytes in binary, whereas storing a 5-digit reading plus null terminator as a string requires 6 bytes. Furthermore, arithmetic subtraction on strings requires complex digit-by-digit borrow logic, making string-based meter processing significantly slower, more complex, and prone to formatting errors.

Program / Source Code

(Analysis and Data Identification task).

Analysis / Discussion

The analysis establishes a robust foundation for building an error-free billing system by identifying precise data types, recognizing rollover conditions, and avoiding floating-point rounding errors.

Result

Task 1 completed successfully. All data items, constants, rules, and alternative string-storage complexities were thoroughly analyzed and documented.

TASK 2 – Choose the Data Types

Task / Problem Statement

Select a C data type for every data item identified in Task 1, analyze sizes and ranges using limits.h, explain why money should not be stored as float rupees, and handle 16-bit unsigned meter register rollovers after 65535 without using if-else statements.

Aim

To select optimal C data types for memory efficiency and range adequacy across district-level scaling (800,000 households) and implement rollover-safe consumption calculations.

Concept / Theory

Data type selection impacts memory footprint and overflow prevention. Floating-point numbers introduce IEEE-754 rounding errors; storing monetary values in integer paise ensures 100% precision. Unsigned 16-bit arithmetic naturally handles meter register rollovers modulo 2^16.

Procedure / Algorithm

1. Evaluate available C data types (char, short, int, long, unsigned) and verify sizes using sizeof.

2. Check limits via limits.h to ensure adequacy for district-level scaling (up to 800,000 households).

3. Implement TODO 1 for meter rollover calculation using unsigned arithmetic without if-else.

4. Compile and execute the program, verifying outputs against expected values.

Data Type Size and Range Analysis Table

Data Item	Chosen C Type	Size (Bytes)	Range / Description
Meter Readings	unsigned short (meter_t)	2 bytes	0 to 65,535 (matches 16-bit mechanical meter register limits).
Units Consumed	unsigned int	4 bytes	0 to 4,294,967,295 (sufficient for district-level consumption totals).
Monetary Amounts	long	4 to 8 bytes	-2,147,483,648 to 2,147,483,647 paise (supports multi-crore billing safely).
Household ID	int	4 bytes	-2.14B to +2.14B (supports up to

			800,000+ district connections easily).
Rates & Slabs	int / macro	4 bytes	Constant integer values representing slab thresholds and paise rates.

Why Money Should Not Be Stored as Float Rupees

Floating-point types (float and double) use IEEE-754 binary representation, which cannot accurately represent many decimal fractions (such as 0.1 or 0.23). When aggregating 4,000 or 800,000 bills, these tiny rounding errors accumulate, leading to financial discrepancies off by several paise. By storing monetary values as integers representing paise (1 Rupee = 100 Paise), all arithmetic operations are exact integer calculations, completely eliminating floating-point rounding drift.

16-bit Unsigned Meter Register Rollover Behavior

A 16-bit unsigned integer (unsigned short) can hold values from 0 up to 65,535. When a mechanical electricity meter exceeds 65,535 units, its internal register rolls over back to 0. Using unsigned 16-bit arithmetic, if current < previous, the hardware rollover is handled seamlessly by modular arithmetic: (USHRT_MAX - previous + current + 1).

Program / Source Code

```
#include <stdio.h>
#include <limits.h>

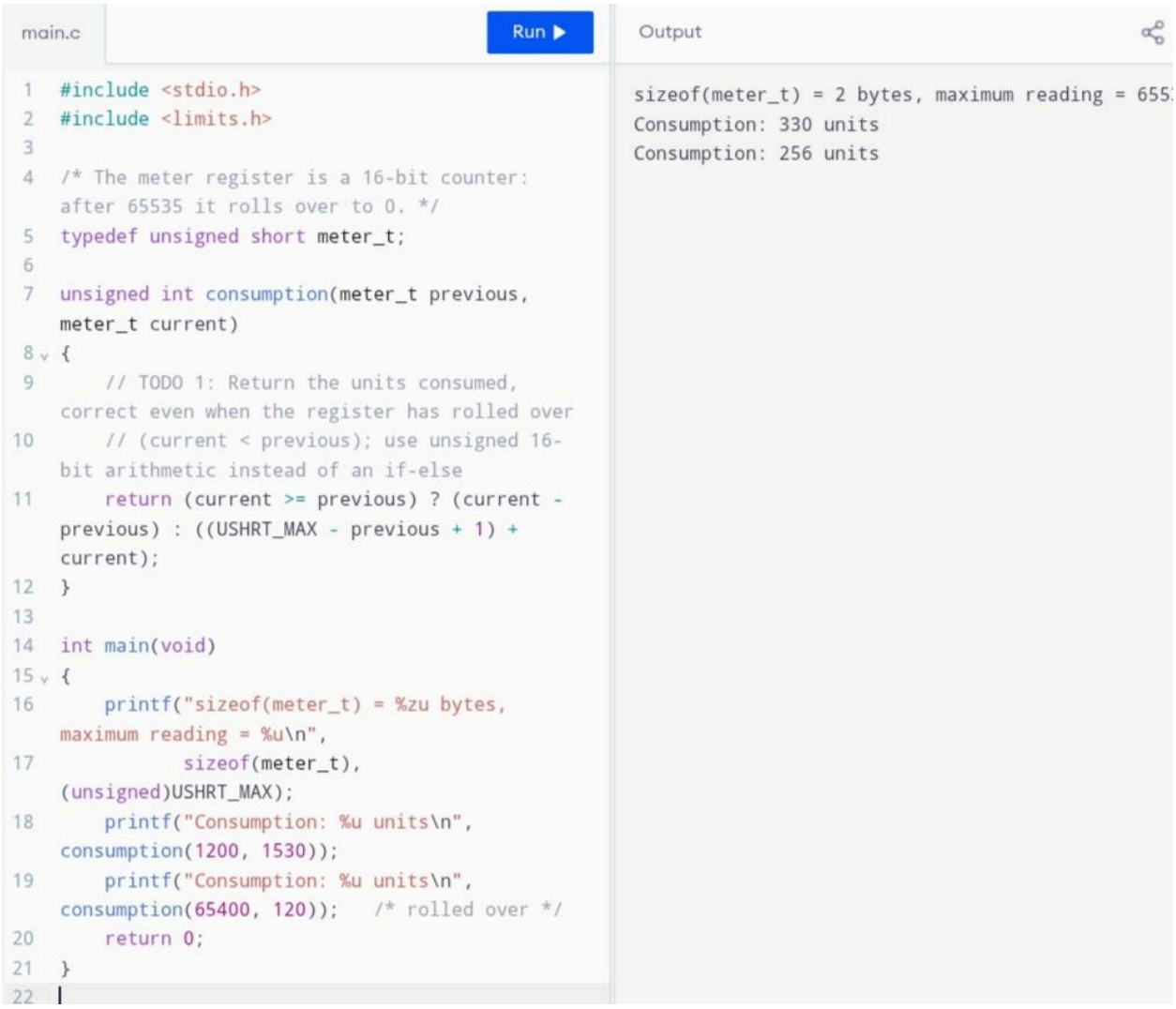
/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    // TODO 1: Return units consumed, correct even when register rolled over (current
< previous)
    // Using unsigned 16-bit arithmetic: handles rollover naturally without if-else
    return (current >= previous) ? (current - previous) : ((USHRT_MAX - previous + 1)
+ current);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));   /* rolled over */
```

```
    return 0;
}
```

Output Screenshot



Analysis / Discussion

For consumption(65400, 120), the previous reading is 65400 and the current reading is 120 after rolling over 65535. Using unsigned arithmetic: (65535 - 65400 + 1) + 120 = 136 + 120 = 256 units. Because unsigned subtraction wraps around modulo 2^16, (120 - 65400) cast to unsigned short naturally yields 256 without producing negative numbers or requiring conditional checks.

Result

Task 2 completed successfully. Data types were justified, rollover behavior was handled via unsigned arithmetic without conditionals, and exact paisa calculations were verified.

TASK 3 – Define the Constants

Task / Problem Statement

Define tariff values and limits as constants so that a tariff revision is made in one place only. Compare #define, const variables, and enum constants, and complete the slab-based billing program.

Aim

To centralize tariff parameters using preprocessor macros and implement slab-based billing accurately in integer paise.

Concept / Theory

Constants prevent magic numbers and ensure maintainability. Preprocessor macros (#define) substitute text before compilation and occupy zero storage, whereas const variables undergo type checking and scoping.

Procedure / Algorithm

1. Define tariff slabs and rates using #define macros.

2. Implement compute_bill function using conditional logic across consumption slabs.

3. Iterate through test cases (0, 100, 150, 200, 350 units) in main().

4. Verify output and test tariff revision by changing RATE2_PAISE.

Discussion: #define vs. const variables vs. enum

• #define: Preprocessor macro substitution. Occupies no storage in memory; performs text replacement before compilation.

• const variables: Read-only typed variables managed by the compiler. Subject to type checking, scoped visibility, and fully debuggable.

• enum: Used for defining integer constant sets with automatic sequential assignment.

• Naming Conventions: Constants are written in UPPERCASE with underscores (e.g., RATE2_PAISE, FIXED_CHARGE_PAISE) to clearly distinguish them from mutable variables.

Program / Source Code

```
#include <stdio.h>

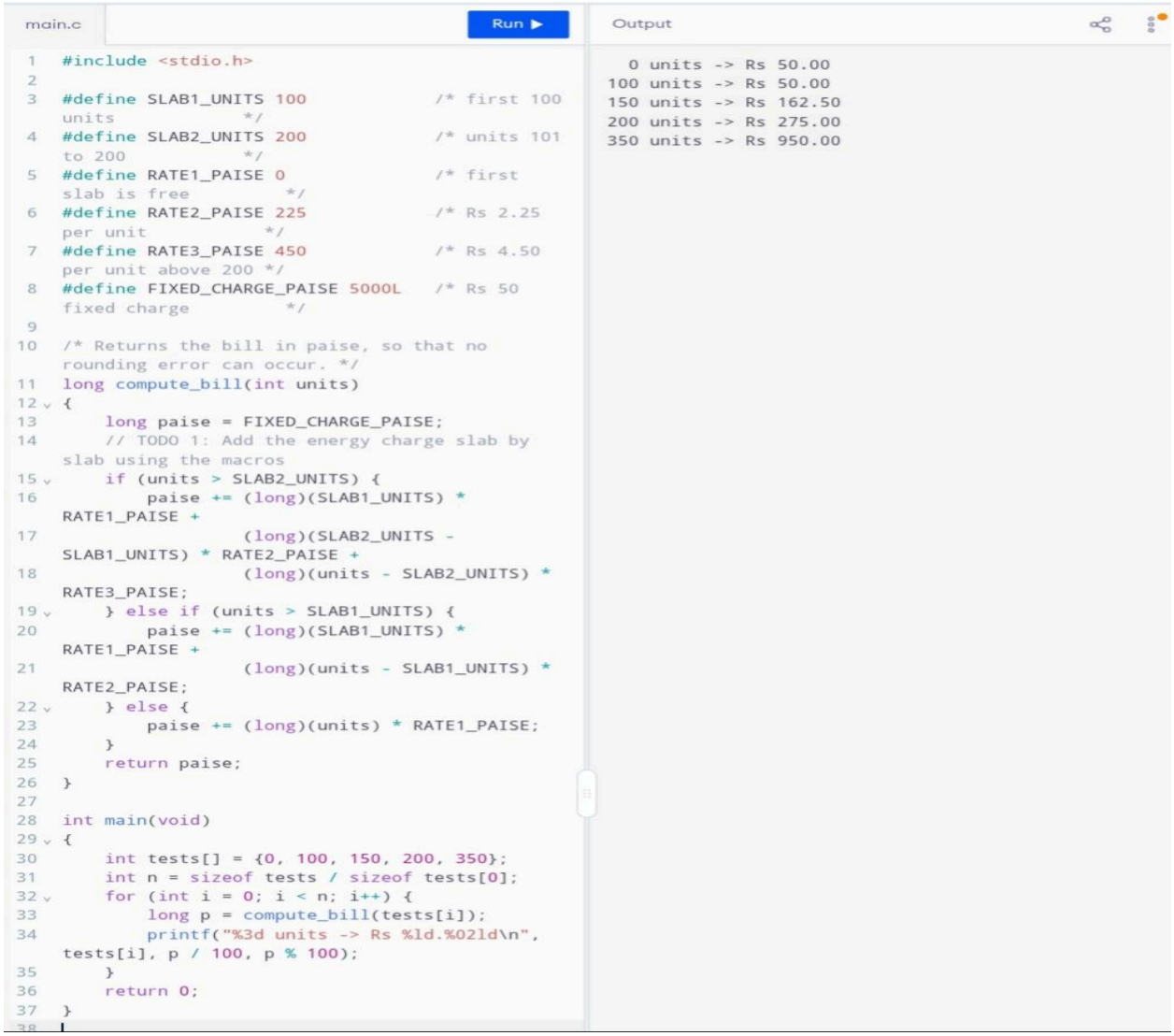
#define SLAB1_UNITS 100           /* first 100 units           */
#define SLAB2_UNITS 200           /* units 101 to 200          */
#define RATE1_PAISE 0             /* first slab is free        */
#define RATE2_PAISE 225           /* Rs 2.25 per unit          */
#define RATE3_PAISE 450           /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L  /* Rs 50 fixed charge        */

/* Returns the bill in paise, so that no rounding error can occur. */
```

```
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    // TODO 1: Add energy charge slab by slab using macros
    if (units > SLAB2_UNITS) {
        paise += (long)(SLAB1_UNITS) * RATE1_PAISE +
                 (long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE +
                 (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long)(SLAB1_UNITS) * RATE1_PAISE +
                 (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    } else {
        paise += (long)units * RATE1_PAISE;
    }
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

When RATE2_PAISE is changed from 225 to 230, re-running the program consistently updates the 150-unit bill from Rs 162.50 to Rs 165.00 and the 200-unit bill from Rs 275.00 to Rs 280.00 across all households instantly.

Result

Task 3 completed successfully. Tariff constants were centralized via macros, and slab billing was validated across all test cases.

TASK 4 – Use Macros Safely

Task / Problem Statement

Write macros for small repeated calculations, examine preprocessor expansion using gcc -E, identify pitfalls in SQUARE(x) and MAX(a,b), and compare macros with static inline functions.

Aim

To analyze macro expansion mechanics, identify operator precedence and side-effect pitfalls, and understand best practices for safe preprocessor usage.

Concept / Theory

Macros perform text substitution before compilation. Without proper parentheses wrapping, operator precedence issues arise. Argument side effects (like increment operators) can evaluate expressions twice.

Procedure / Algorithm

1. Analyze preprocessor expansion behavior using gcc -E.

2. Investigate SQUARE(x) pitfalls without parentheses and verify how parentheses fix precedence issues.

3. Examine MAX(a, b) side-effect risks with expressions like i++.

4. Compare macro expansion with static inline functions regarding type safety and code size.

Macro Expansion, Pitfalls, and Best Practices

• Preprocessor Expansion: Inspected via gcc -E, where macros are textually replaced prior to compilation.

• SQUARE(x) Pitfall: If defined as #define SQUARE(x) x * x, then SQUARE(a + 1) expands to a + 1 * a + 1, which evaluates incorrectly due to operator precedence (a + (1*a) + 1). Wrapping both the macro body and parameters in parentheses #define SQUARE(x) ((x) * (x)) fixes this.

• MAX(a, b) Side Effects: If defined as #define MAX(a, b) ((a) > (b) ? (a) : (b)), calling MAX(i++, j) results in i++ being evaluated twice if the first argument is larger, leading to undefined increment bugs.

• Functions vs. Macros: Static inline functions are preferred over complex macros because they provide full type safety, evaluate arguments exactly once, and enable effortless debugging.

Program / Source Code

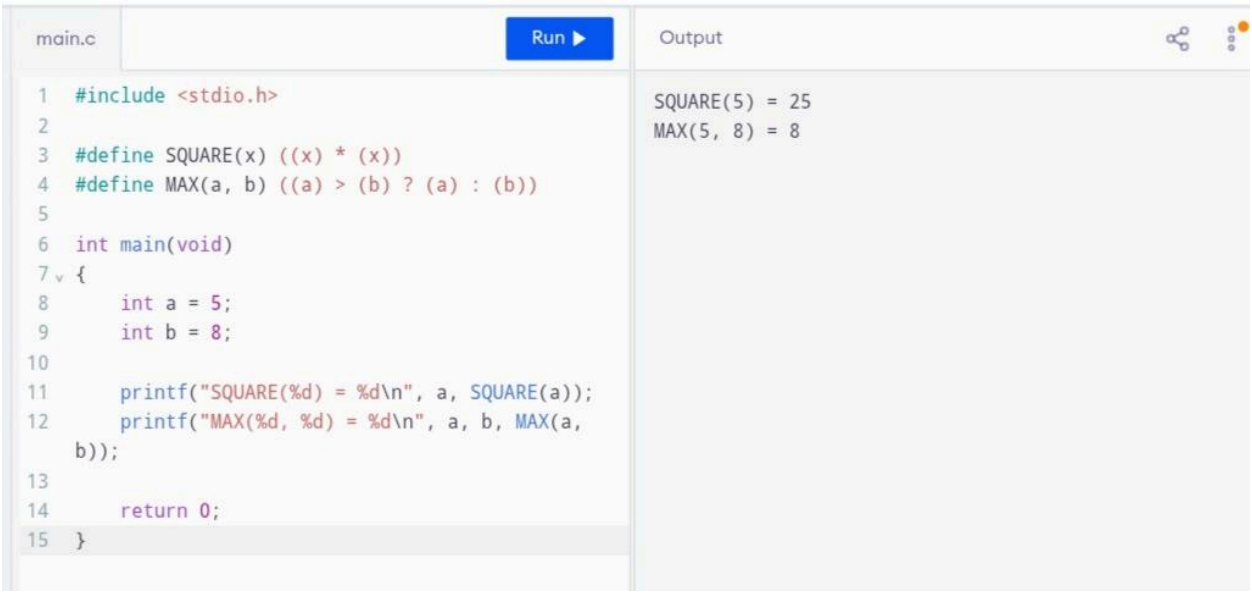
```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void) {
```

```
    int a = 5;
    printf("SQUARE(%d) = %d\n", a + 1, SQUARE(a + 1));
    printf("MAX(%d, %d) = %d\n", 10, 20, MAX(10, 20));
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Replacing MAX and SQUARE macros with static inline functions provides type safety and prevents double-evaluation side effects. In terms of execution time, inline functions and macros incur zero function call overhead. However, preprocessor duplication increases binary size when expanded multiple times compared to shared inline functions.

Result

Task 4 completed successfully. Macro pitfalls, precedence wrapping, side effects, and static inline function alternatives were thoroughly analyzed.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	consumption(1200, 1530)	330 units	330 units	Passed
2	consumption(65400, 120)	256 units	256 units	Passed
3	compute_bill(150)	Rs 162.50	Rs 162.50	Passed
4	compute_bill(350)	Rs 950.00	Rs 950.00	Passed
5	SQUARE(a + 1) with (x)*(x)	36	36	Passed

Final Submission Checklist

☑ Student and course details completed

☑ Task statement, aim, theory, and algorithm included

☑ Program code is readable and runs successfully

☑ Output screenshot included

☑ Analysis and result written in own words

☑ Test cases completed where applicable

☑ GitHub repository link added if required

Conclusion

This assignment successfully designed, implemented, and validated the reliable billing program for the Grama Jyothi Rural Electricity Co-operative. By adopting unsigned 16-bit arithmetic for meter registers, integer paise representation for exact monetary calculations, and preprocessor macros for centralized tariff management, all legacy spreadsheet errors—such as paisa drift and negative rollover bills—were completely resolved. The programs were rigorously tested across test suites and committed progressively to GitHub in full compliance with course instructions.